```
  sourcemap: true,
  outdir: ["app/assets/builds"]
})
```

And you would call that with `node esconfig.js`.

If you want to extend esbuild, it allows you to create and use plugins that add different features, such as the ability to load different file types. I'm not going to talk about creating plugins here (see https://esbuild.github.io/plugins/#concepts), but there's a growing ecosystem of them.

The esbuild team maintains a list of community plugins at https://github.com/esbuild/community-plugins, but doesn't recognize any of them as "official" the way that, say, Tailwind has official plugins.

In order to use esbuild plugins, you must be invoking esbuild via the configuration file. The normal process is to require in the plugin and then reference it as an argument to the `build` method. For example, if we wanted to include CoffeeScript in our application, we might try the esbuild-coffeescript plugin[59][60] as follows:

```
const coffeeScriptPlugin = require('esbuild-coffeescript');

require('esbuild').build({
  entryPoints: ["app/javascript/*.*],
  bundle: true,
  sourcemap: true,
  outdir: ["app/assets/builds"],
  plugins: [coffeeScriptPlugin]
})
```

At which point any CoffeeScript in our codebase would be properly converted to JavaScript and bundled.

There are plugins for most major front-end programming file formats, including Vue, Svelte, PostCSS, and Sass, as well as other tooling including a plugin that auto-imports Stimulus controllers, which would be a replacement for the manifest we've been generating.